Introduction

This document describes the environment to be used for reinforcement learning tasks related to entanglement distribution and experiment scheduling. We call the entity which interacts with the environment the *agent*.

Environment Properties

This *environment* represents a network of quantum nodes each containing 1 or more quantum memories. Some of these nodes are connected by physical links, each representing a shared entanglement pair between the two nodes. For those nodes that are not connected by a physical link, a *virtual link* between the two nodes can be generated by the agent using entanglement swapping operations across nodes along their shortest path with respect to the physical links.

The network is a graph $G = (N, E)$ of quantum nodes N and physical links E. The network transitions in discrete time steps. At the start of a given time step t, each physical link attempts to activate its entanglement pair. This may fail, meaning that some edges of E may not be active at time t. Hence, the graph at time t is $G_t = (N, E_t)$, where $E_t \subseteq E$. A link can be activated with a probability of p_{link}.

Hence a network is characterized by the parameters

- $G = (N, E)$ given by an adjacency matrix where each entry is a value L, the optical link length between the two nodes.
- N_i contains a set of n_i quantum memories.

The physical links are either **active** or **inactive**. When inactive, a physical link attempts to generate with probability based on the optical link length L between the two nodes, where

$$p_{\text{link}} = e^{-\gamma L}$$

where γ is an attenuation coefficient. These links are active for a maximum of m^* time steps, at which point they are deactivated, and on the next time step, they attempt to generate again with probability p_{link}.

The state space of the environment is a set of matrices containing:

- E^t, an $|N| \times |N|$ matrix where each entry $E^t_{i,j} = -1$ if the link between nodes i and j is inactive, and $E^t_{i,j} = m$ if the link is active with an "age" of $m \geq 0$ time steps. When $E^t_{i,j} = m^*$, the link is deactivated in the next time step.
- A vector $M = |N|$ where M_i is the number of quantum memories remaining for node N_i.
- A request queue Q of size $|Q_0|$ containing the experiments waiting to be run.

The action space of the environment is the following:

- **Entanglement swap**: Select two adjacent links (i, j) and (j, k) and perform a BSM to create a virtual link (i, k) with age $\max\left(E^t_{i,j}, E^t_{j,k}\right)$. This virtual link can be used for further entanglement swaps or to be used for an experiment.

Intermediate physical links are deactivated in the process (set to -1 in the matrix).

- **Map Experiment**: Assign one expereiment topology to a subgraph of active links (physical or virtual). All edges must be active.
- **Wait**: Do nothing.

Each timestep flows as follows:

1. Environment update
   - Increment the age of each active link by 1.
   - Attempt to activate each inactive link with success probability p_{link}, setting their age to 0 if successful.
   - If an active link's age is greater than m^*, set it to -1 to be activated next step.
2. Action is taken by the agent and the environment is updated:
   - If the action is **Entanglement swap**, update the edge matrix E^t to the max age of the two adjacent physical links, and update the memory vector M.
   - If the action is **Map Experiment**, remove the experiment from Q

Policy Architecture

Pros and Cons

Pros

High Fidelity: Captures fundamental quantum networking constraints, including memory bottlenecks, link attenuation (p_{link}"), and decoherence (m^*).

Flexibility: Supports arbitrary physical topologies and diverse experiment requests via subgraph isomorphism.

Standard MDP Structure: Matrix-based states and discrete actions align with established reinforcement learning frameworks and algorithms.

Resource Management: Explicitly models memory occupation, forcing the agent to balance long-range entanglement against local resource exhaustion.

Cons

Scalability: The $O(N^2)$ matrix representation leads to sparse data structures and increased computational overhead as node count increases.

Action Space Complexity: Identifying all valid experiment mappings (subgraph isomorphism) is computationally intensive for large networks or complex topologies.

Discrete Decoherence: The hard cutoff at m m^* simplifies the continuous nature of quantum decoherence and Pauli noise.

Local Optima Risk: High p_{link} failure rates may lead to sparse graphs where the agent struggles to find valid paths, slowing initial training convergence.

Implementation Todos:

☑ Implement operations on edges and memories

- [x] Implement experiment assignment
- [ ] Implement policy
  - [ ] Explore policy options, GNN, etc.
  - [ ] Implement policy training
    - [ ] Define action masking
    - [ ] Define reward function

Calculation of Size of Action Space

Let the number of nodes be N. Let the total number of physical links be $|E|$.

The agent can do one of the following:
1. Wait
2. Establish a virtual link
3. Assign an experiment

The cardinality of the action space is 1 + maximum number of possible virtual links to choose from + maximum number of experiment mappings to choose from

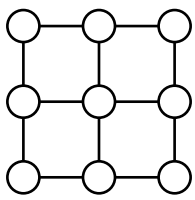
The maximum number of virtual links to choose from is the total number of possible node-node pairs minus the number of physical links, i.e.

$$\text{number of virtual links} = |V| = N(N-1)/2 - |E|$$

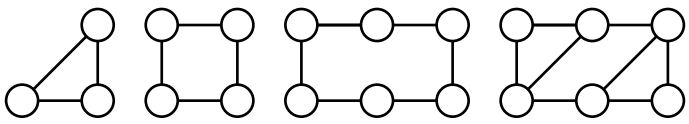
For each experiment we count the number of permutation-invariant subisomorphisms of the experiment topology on the complete graph K_N, i.e. the complete, maximum-connectivity graph on N nodes. For experiment X_j, let $\mathrm{Map}_{K_N}(X_j)$ be the number of such isomorphisms of X_j's topology on K_N. Then the total size of the action space is

$$|A| = 1 + |V| + \sum_{j=1}^{n} \mathrm{Map}_{K_N}(X_j)$$

Let's consider a concrete example of an 3 by 3 grid network of nodes, each with a number of quantum memories equal to the degree of the node. Then $N = 9$ and $|E| = 12$, and the total number of possible virtual links is $|V| = 24$.

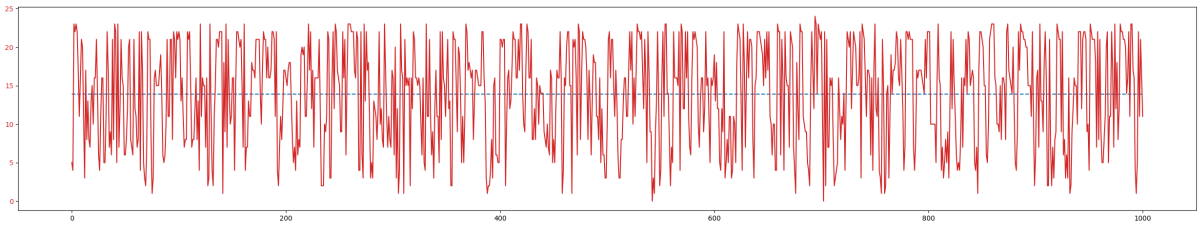


Let's say we have four experiments with the following topologies:

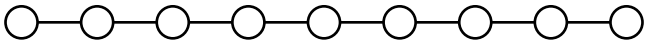


Then there are 84 mappings for the first topology, 378 for the second topology, 5040 for the third, and 15120 for the fourth. Then the full action space is $1 + 24 + 84 + 378 + 5040 + 15120 = 20{,}622$ actions, which is huge.

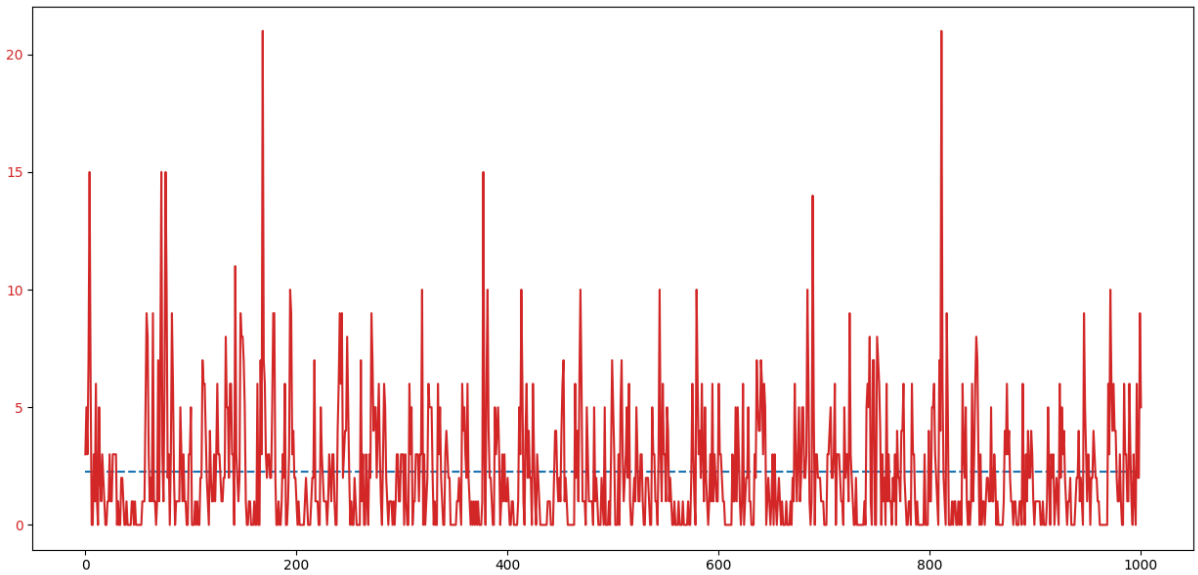
However, maybe the full K_N is too large than what will realistically happen under the dynamics of the network. Let's plot the number of virtual links possible to assign over time steps for the grid network, also plot the ratio of this number to the total number of possible links in K_N.



We see the average number of virtual links possible to assign is 13.89 out of a total of 24 possible virtual (non-physical) links or about 57%. But sometimes all of them *are* possible, which makes things difficult. Some setups where the ratio is smaller is if the ratio of physical links to to total links is small. For example, let's consider a linear chain of 9 nodes:



Here's the graph for the number of virtual links:



The average here is 2.16 out of a total of 24 possible virtual links, or about 9%. So clearly, most of the time we only need to consider a small subset of the possible virtual links.